Program Structures and Algorithms
Spring 2025

NAME: Xinduo Fan

NUID: 002059620

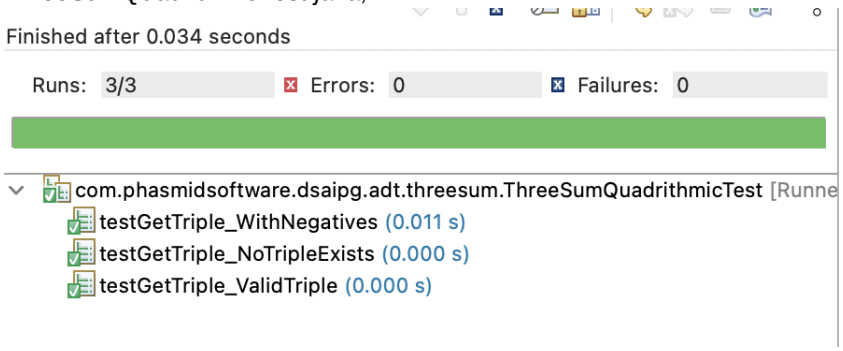
GITHUB LINK: https://github.com/XDagain-jpg/DSAIPG

Assignment 2 (3-SUM) Task:
Submit (in your own **FORK**--see instructions elsewhere--include the source code and the unit tests of course):
(a) evidence (screenshot) of your unit tests running (try to show the actual unit test code as well as the green strip);
(b) a spreadsheet showing your timing observations--using the doubling method for at least **five values of N**--for each of the algorithms (**include cubic**); Timing should be performed either with an actual stopwatch (e.g. your iPhone) or using the **Stopwatch class in the repository.**
(c) your brief explanation of why the quadratic method(s) work.

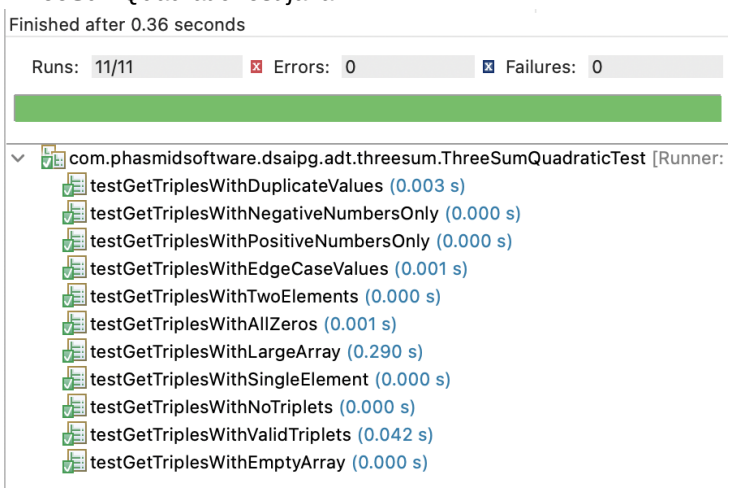
For the **codes:**
please see my **FORK** for more details.
For the **spreadsheet:**
please see **assignment2_3sum_benchmark_result.xlsx** for more details.

Unit Test Screenshots:

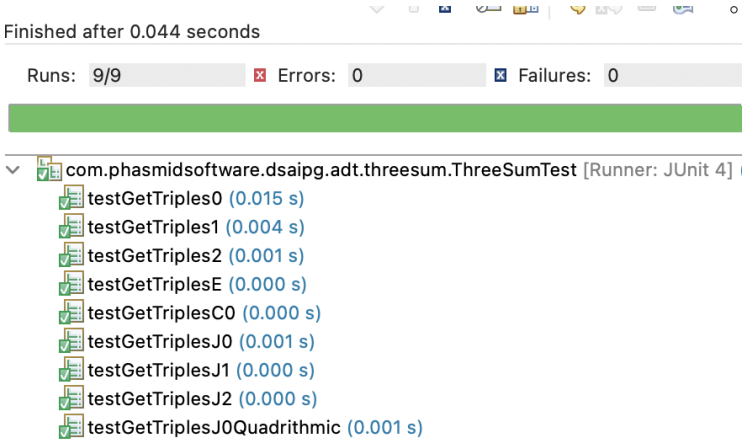
ThreeSumQuadrithmicTest.java,



ThreeSumQuadraticTest.java



ThreeSumTest.java



Brief Explanation of why the quadratic method(s) work

```
/**
 * Retrieves an array of unique Triples. Each Triple represents a unique combination of three integers from
 * the source array that sum to zero.
 *
 * @return an array of distinct Triples, sorted in natural order, where each Triple satisfies the condition that
 * the sum of its three integers is zero.
 */
public Triple[] getTriples() {
    List<Triple> triples = new ArrayList<>();
    for (int i = 0; i < length; i++) triples.addAll(getTriples(i));
    Collections.sort(triples);
    return triples.stream().distinct().toArray(Triple[]::new);
}
```

From this outer loop in getTriples() function, we can see that it iterates through the whole array, takes N steps.

```
/**
 * Get a list of Triples such that the middle index is the given value j.
 *
 * @param j the index of the middle value.
 * @return a Triple such that
 */
List<Triple> getTriples(int j) {
    List<Triple> triples = new ArrayList<>();
   // TO BE IMPLEMENTED  : for each candidate, test if a[i] + a[j] + a[k] = 0.

    if (length < 3) return triples;

    int i = 0, k = length-1;

    while (i < j && j < k) {
        int sum = a[i] + a[j] + a[k];
        if (sum == 0) {
            triples.add(new Triple(a[i] , a[j] , a[k]));
            i++;k--;
        }else if (sum < 0 ) {
            i++;
        }else if (sum > 0 ) {
            k--;
        }
    }

    //throw new RuntimeException("implementation missing");
    return triples;
}
```

This means that within each step (for each i in outer loop and each input int j in this function) we need to test all possible i and k candidates against input j, such that we find all a[i] + a[j] + a[k] == 0. To achieve a quadratic complexity, we need to loop through the sorted array without nested loop in this function. Hence, we use two pointers i and k to iterate towards each other, where i starts front and k

starts at the end. Since we know j is a fixed input, we let it serve as the middle element in the triplet and remember in the outer function, this input j iterates over all valid indices once, every number in the array acts as the middle element at least once. We also know that the array is sorted, and we always move i forward for one index when the sum<0 and k backward for one index when the sum>0, no number is skipped without being checked against j. The two-pointer search ensures that all valid sums are considered without missing any valid pair. And the loop stops when i or k reaches j, meaning that all possible sorted triples that have j at the middle position have been tested. And this while loop will cover at most N iterations, for i or k to reach j.